

✓ Load and Explore the Dataset

We will use the Iris dataset from `sklearn.datasets`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

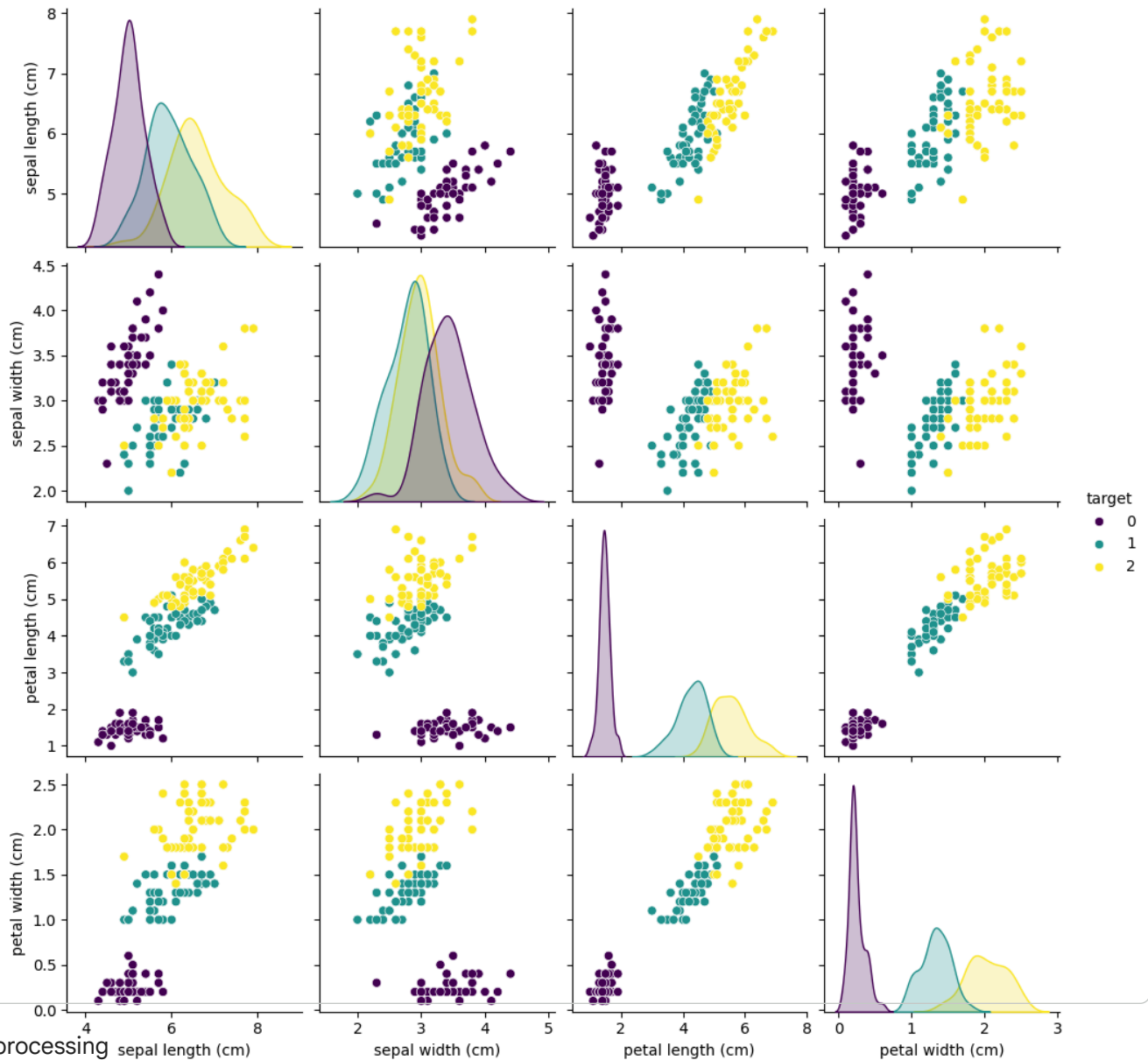
# Load dataset
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['target'] = iris.target

# Explore missing values
print("Missing values:\n", df.isnull().sum())

# Visualize feature distributions
sns.pairplot(df, hue='target', palette='viridis')
plt.show()

display(df.head())
```

```
Missing values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64
```



Preprocessing

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2
0	0	0	0	0

Splitting the data and applying feature scaling.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop('target', axis=1)
y = df['target']

# Split data (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Hyperparameter Tuning (K-selection)

Testing different k values using cross-validation.

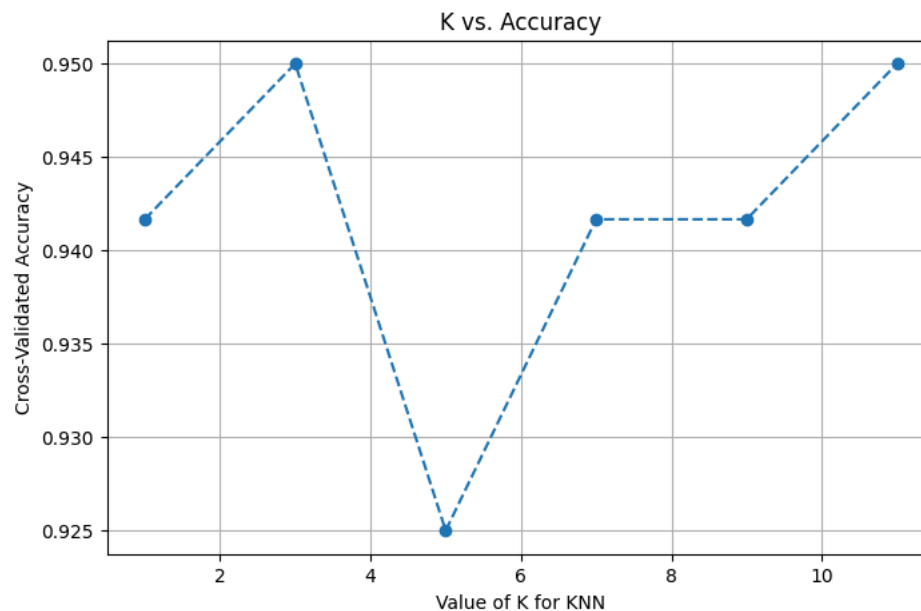
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

k_values = [1, 3, 5, 7, 9, 11]
cv_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train_scaled, y_train, cv=5)
    cv_scores.append(scores.mean())

# Plot k vs accuracy
plt.figure(figsize=(8, 5))
plt.plot(k_values, cv_scores, marker='o', linestyle='dashed')
plt.xlabel('Value of K for KNN')
plt.ylabel('Cross-Validated Accuracy')
plt.title('K vs. Accuracy')
plt.grid(True)
plt.show()

optimal_k = k_values[np.argmax(cv_scores)]
print(f"Optimal K identified: {optimal_k}")
```



Optimal K identified: 3

Final Model Evaluation

Evaluating the best model on the test set.

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Initialize and train with optimal k
final_model = KNeighborsClassifier(n_neighbors=optimal_k)
final_model.fit(X_train_scaled, y_train)

# Predictions
y_pred = final_model.predict(X_test_scaled)

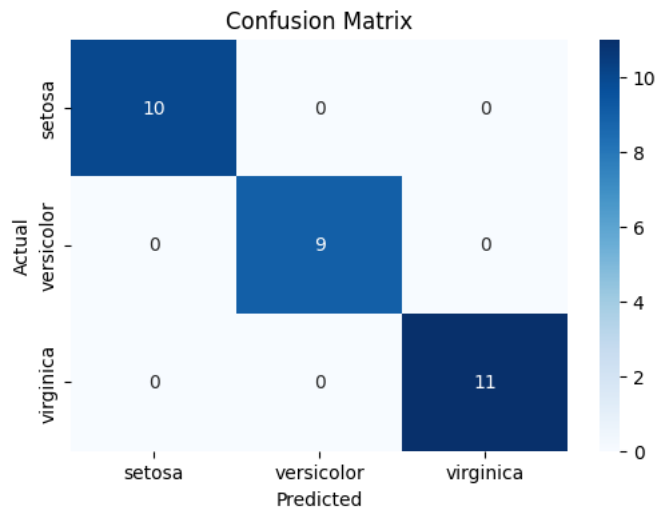
# Metrics
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=iris.target_names))

# Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Predicted')
```

```
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

Accuracy: 1.0

Classification Report:	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30



Test on New Samples

Predicting classes for unseen data points.

```
new_samples = np.array([
    [5.1, 3.5, 1.4, 0.2], # Expected: setosa
    [6.5, 3.0, 5.2, 2.0] # Expected: virginica
])

# Convert to DataFrame to match the format used during training and avoid warnings
new_samples_df = pd.DataFrame(new_samples, columns=iris.feature_names)

# Scaling is required for new points too
new_samples_scaled = scaler.transform(new_samples_df)
predictions = final_model.predict(new_samples_scaled)

for i, pred in enumerate(predictions):
    print(f"Sample {i+1} prediction: {iris.target_names[pred]}")

Sample 1 prediction: setosa
Sample 2 prediction: virginica
```

Detailed Performance Metrics

Beyond accuracy, we can look at precision, recall, and the F1-score to understand how the model performs on individual classes. This is especially useful if classes are imbalanced.

```
from sklearn.metrics import precision_score, recall_score, f1_score

# Calculate metrics using the test set
precision = precision_score(y_test, y_pred, average=None)
recall = recall_score(y_test, y_pred, average=None)
f1 = f1_score(y_test, y_pred, average=None)

metrics_df = pd.DataFrame({
```

```
'Class': iris.target_names,  
'Precision': precision,  
'Recall': recall,  
'F1-Score': f1  
})  
  
display(metrics_df)
```